

CAP Theorem - CA Databases (Consistency + Availability)

1. What is a CA System?

CA means:

None

C = Consistency

A = Availability

System provides:

- Latest correct data (Consistency)
- Every request gets response (Availability)

But:

None

No Partition Tolerance

If network split happens, guarantees break.

2. Important Real-World Clarification

In true distributed systems, network partitions eventually happen.

So pure **CA distributed databases are uncommon / mostly theoretical.**

Because once partition occurs, system must choose:

None

Consistency OR Availability

Cannot keep both perfectly.

3. Where CA Exists Practically

CA is usually seen in:

A. Single-Node Databases

No distributed partition issue.

Examples:

- MySQL single instance
- PostgreSQL single instance

These can give:

- Strong consistency
- High local availability (if machine healthy)

B. Distributed Systems Assuming Reliable Network

Some systems behave CA under normal operation until partition occurs.

4. Example

Single primary database server:

None

Write ₹500

Immediate read returns ₹500

Server responds normally

Consistent + available while server/network healthy.

5. Why CA Is Less Practical at Scale

In multi-region / multi-node systems:

- Links fail
- Routers fail
- Packet loss happens
- Data centers isolate

Then partition tolerance becomes mandatory.

So systems usually become:

None

CP or AP

6. MySQL / PostgreSQL Clarification

Your statement needs refinement.

MySQL and PostgreSQL are not inherently “CA databases” in all deployments.

Depends on architecture:

Single Node

Mostly CA-like behavior.

Primary + Replica Setup

During partition:

- Stop writes to preserve consistency → CP-like behavior
- Allow stale reads from replicas → partial AP tradeoff

So CAP depends on deployment choices.

7. When CA Model Fits

Useful when:

None

Single region

Single DB node

Low distribution needs

Strong transactional workloads

Examples:

- Internal ERP
- Small SaaS backend
- Monolith apps
- Admin systems

8. Benefits

Simpler architecture

Strong transactions

Easier debugging

Lower consistency complexity

9. Limitations

Single point of failure (without failover)

Poor geo-distribution

Not resilient to partitions

Harder global scale

10. CAP Comparison

Type	Guarantees	Typical Tradeoff
CA	Consistency + Availability	No partition tolerance

CP	Consistency + Partition tolerance	May reject requests
AP	Availability + Partition tolerance	May serve stale data

11. Interview Answer Example

Q: Are MySQL/PostgreSQL CA databases?

In single-node deployments they behave like CA systems. In distributed replicated deployments, their CAP behavior depends on how failover, reads, and partitions are handled.

12. Short Revision Notes

None

CA =

Consistency + Availability

Best fit:

single-node systems

Distributed real world:

partitions force CP/AP choices

13. One-Line Summary

CA systems provide correct data and responsive service, but in real distributed environments they become impractical because network partitions force a choice between consistency and availability.